

IntelliMeet Pro: AI Meeting Assistant & Workflow Management

Software Re-Engineering

Group: Laiba (22K-5195), Mustafa (22k-4818), Sabina (22k-5198)



Department of SE

Fast School of Computing

Fast National University, Karachi Campus

Table of Contents

1. Introduction.....	3
2. Selected Application.....	3
2.1. Key features.....	3
2.2. Target users.....	3
3. System Exploration.....	3
3.1. System Overview.....	3
3.2. Main functionalities.....	4
4. Reverse Engineering Analysis.....	5
4.1. System Architecture.....	5
4.2. System Modules.....	5
4.3. Database Entities.....	6
5. UML Diagrams.....	6
5.1. Use Case Diagram.....	6
5.2. Domain Model.....	11
5.2.1. Class diagram.....	11
5.2.2. ER diagram.....	12
5.3. Data Flow Diagram	13
5.3.1. Level 0 Context diagram.....	13
5.3.2. Level 1 DFD.....	14
6. Weakness Identification.....	14
7. Re-Engineering Proposal.....	15
8. Code modifications.....	18
9. Conclusion.....	24

1. *Introduction*

Software systems evolve over time, and maintaining their efficiency, scalability, and usability requires systematic improvement. Software re-engineering is a disciplined process that involves analyzing an existing system, understanding its structure and behavior, and improving it without changing its core functionality.

This project focuses on the re-engineering of **IntelliMeet Pro**, an AI-powered meeting automation platform. The system is analyzed through reverse engineering techniques to extract architectural and design knowledge. Based on this understanding, weaknesses are identified and improvements are proposed.

The objective of this report is to:

- Understand the existing system
- Recover system design
- Identify limitations
- Propose re-engineering improvements

2. *Selected Application*

IntelliMeet Pro is a cloud-based AI meeting assistant designed to automate meeting workflows and convert discussions into structured knowledge.

2.1. *Key features*

- Automatic meeting bot joining (Zoom, Google Meet, Teams)
- Audio recording and storage
- Speech-to-text transcription using AI
- Speaker diarization (identifying speakers)
- AI-based meeting summarization
- Action item extraction and assignment
- Semantic search using vector databases
- Slack bot integration for queries
- Integration with Jira, Trello, Asana
- Subscription and billing system

2.2. *Target users*

- Organizations and corporate teams
- Project managers and team leads
- Developers and knowledge workers
- Remote and hybrid teams

3. *System Exploration*

3.1. *System overview*

IntelliMeet Pro transforms traditional meetings into structured, searchable, and actionable insights. The system automatically joins meetings, records audio, processes it using AI, and provides summaries and action items.

It eliminates manual note-taking and improves productivity by bridging the gap between discussions and execution.

3.2. *Main functionalities*

1. User Management

- Registration and login
- Email verification
- Role-based access control (Admin, Member)

2. Workspace Management

- Create workspaces
- Invite members
- Assign roles

3. Meeting Management

- Schedule meetings
- Deploy AI bot
- Record meeting audio

4. Transcription

- Convert speech to text
- Timestamped transcript generation
- Speaker identification

5. Summarization

- AI-generated summaries
- Key points and decisions extraction

6. Action Item Extraction

- Identify tasks from discussions
- Assign tasks to users

7. Semantic Search

- Natural language queries
- Retrieve relevant meeting content

8. Integrations

- Jira, Trello, Asana
- Slack bot interface

9. Subscription System

- Multiple pricing tiers
- Payment processing

4. Reverse Engineering Analysis

4.1. System Architecture

The system follows a Layered Architecture deployed on cloud infrastructure.

1. Presentation Layer

- Web application
- Slack bot interface

2. Application Layer

- .NET backend API
- Business logic handling

3. AI/NLP Layer

- Whisper (transcription)
- GPT (summarization)
- Translation services

4. Data Layer

- SQL database (structured data)
- Azure Blob Storage (audio files)
- Pinecone (vector embeddings)

5. Integration Layer

- External APIs (Zoom, Slack, Stripe, etc.)

4.2. System Modules

The system is divided into the following modules:

1. Authentication Module
2. Workspace Management Module
3. Meeting Management Module
4. Transcription Module
5. Summarization Module
6. Action Item Module
7. Search Module (RAG-based)
8. Integration Module
9. Subscription Module
10. Notification Module

4.3. Database Entities

Key entities include:

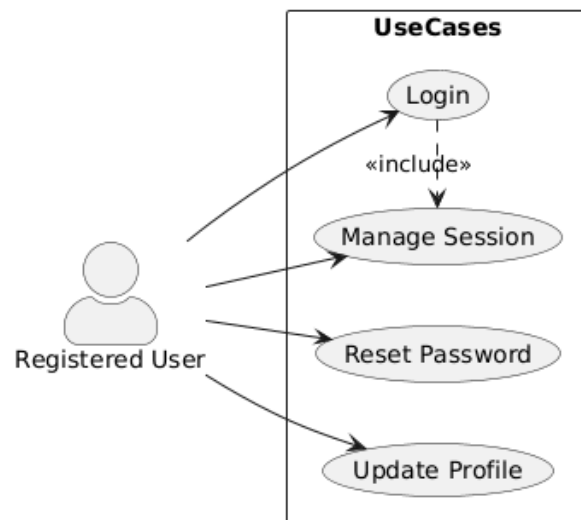
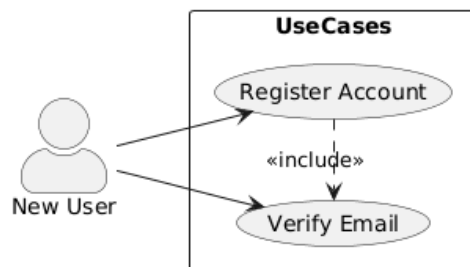
1. **Users** → stores user information
2. **Workspaces** → groups users and meetings
3. **Meetings** → meeting metadata
4. **Transcripts** → speech-to-text data
5. **Summaries** → AI-generated summaries
6. **ActionItems** → extracted tasks
7. **SpeakerProfiles** → voice identity tracking
8. **Integrations** → API tokens
9. **Subscriptions** → billing details

5. UML Diagrams

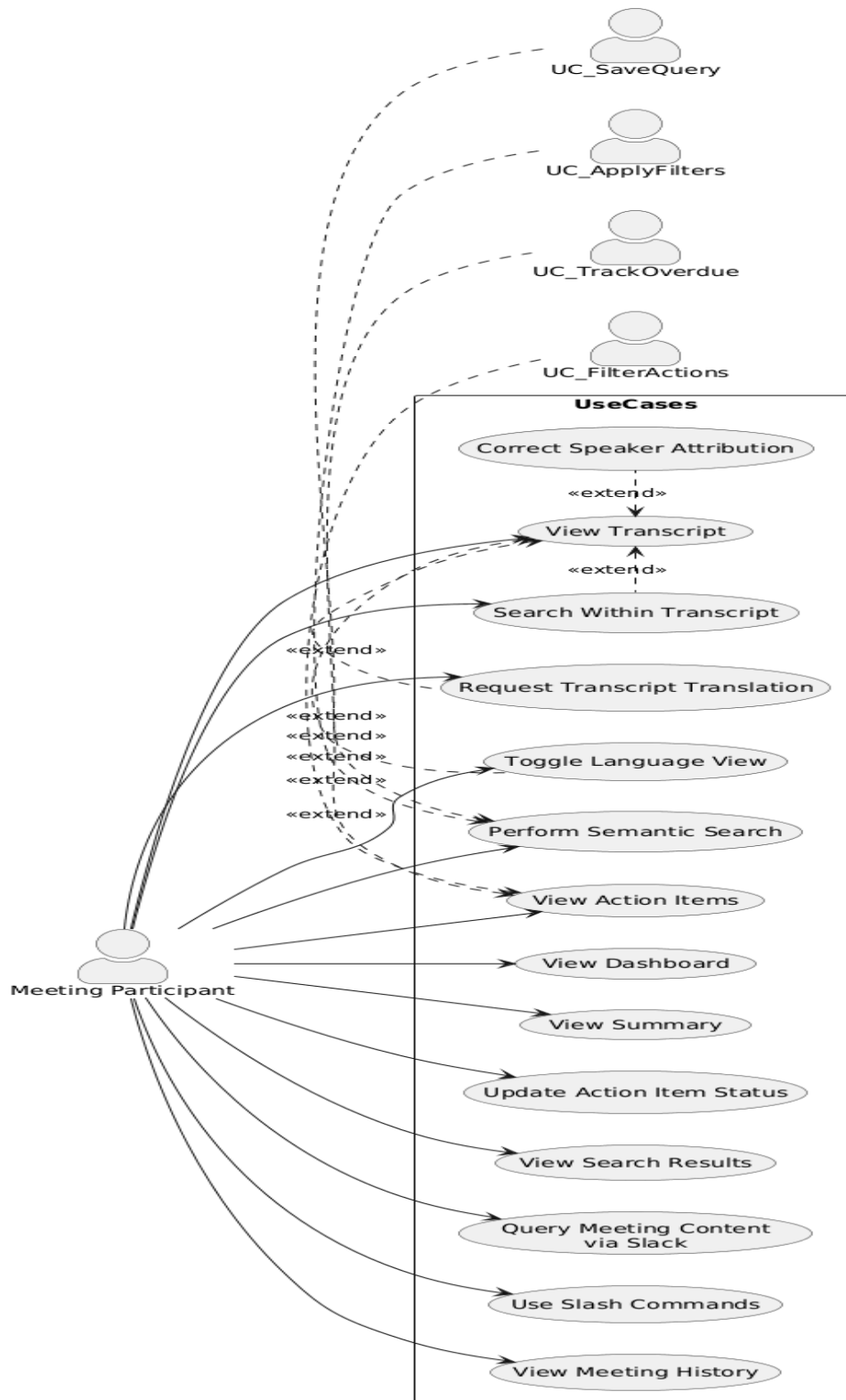
5.1. Use Case Diagram

Actors and Use Cases:

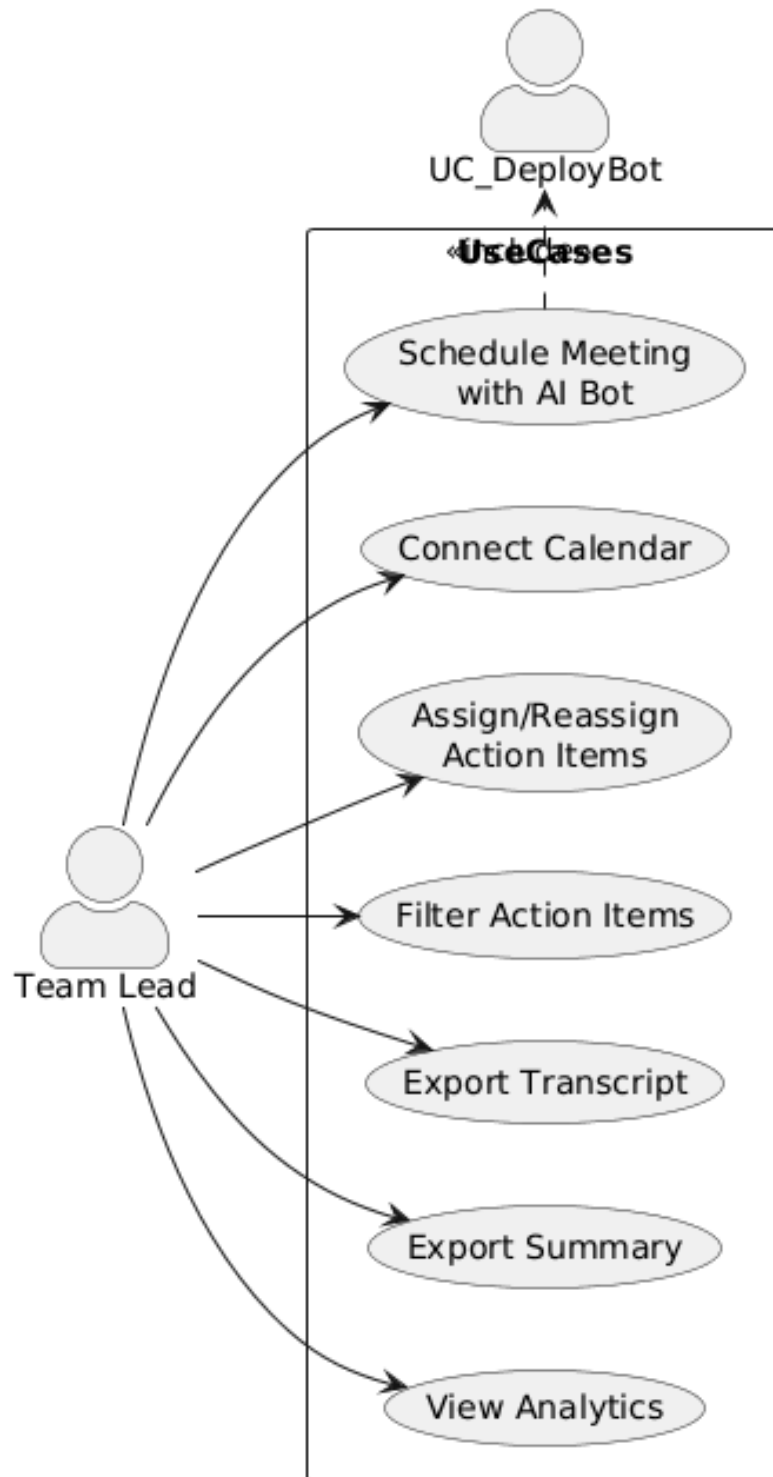
- New User / Registered User: Register/Login



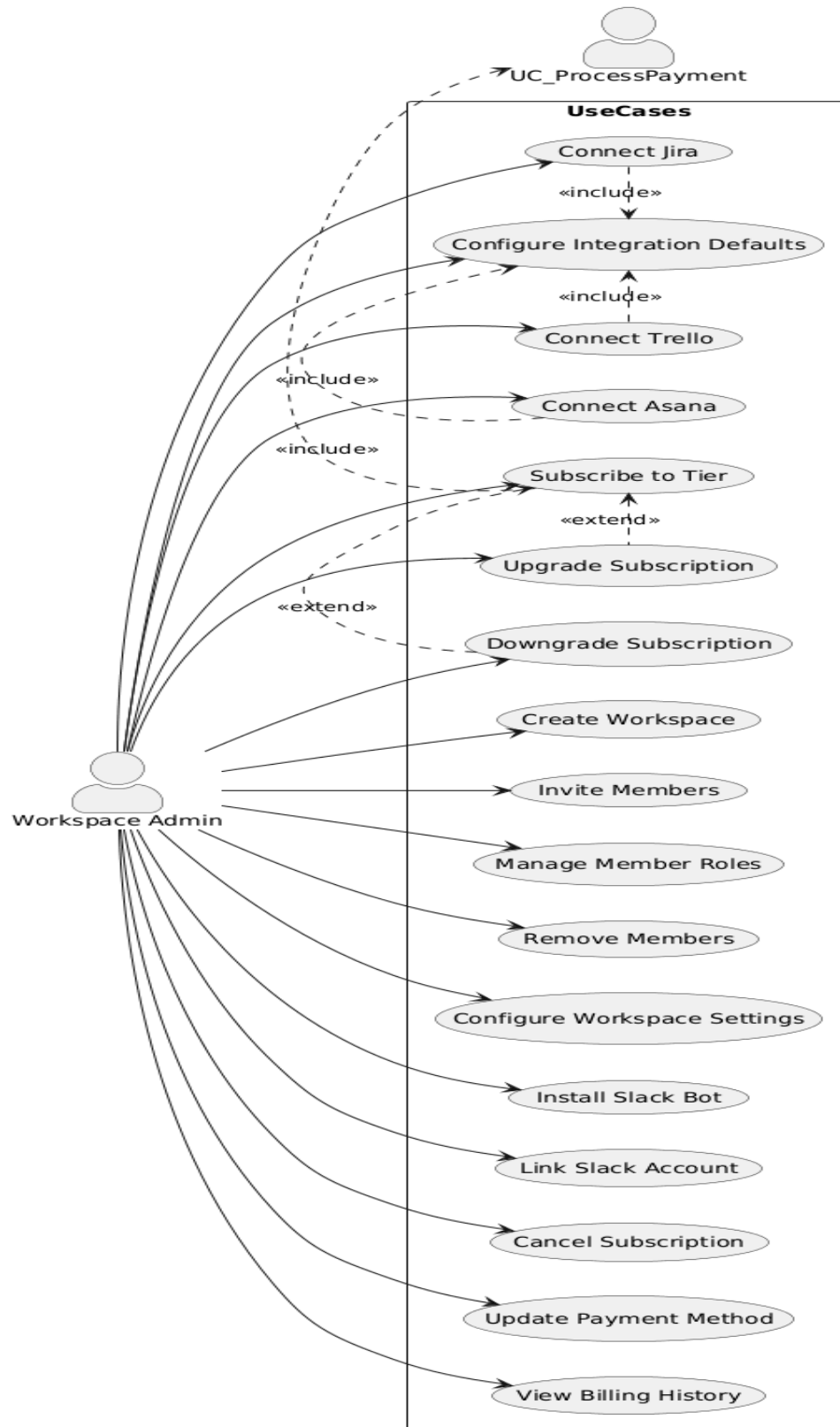
- Meeting Participant: Join Meeting via Bot



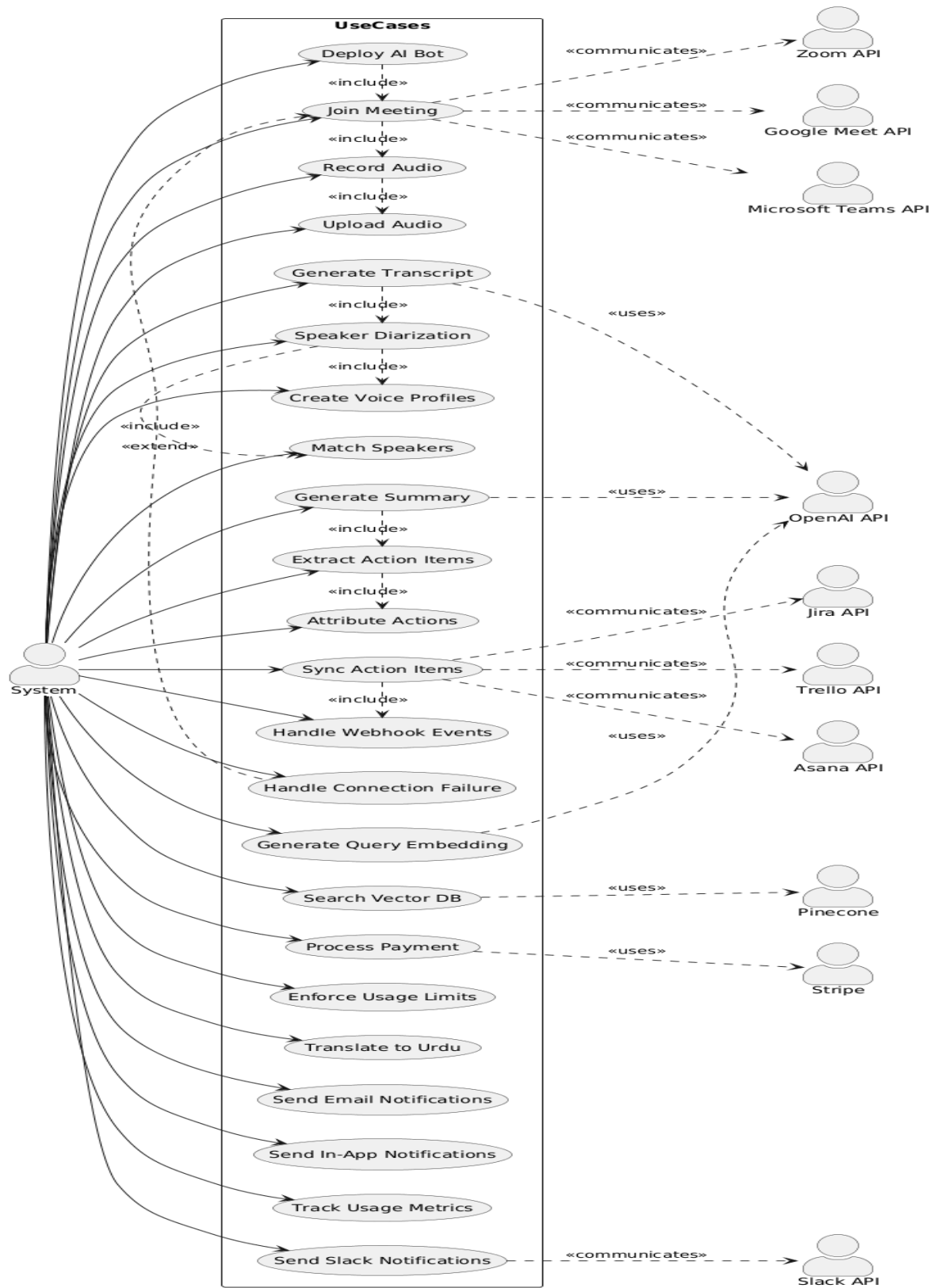
- Team Lead:Schedule Meeting



- Workspace Admin:Create Workspace



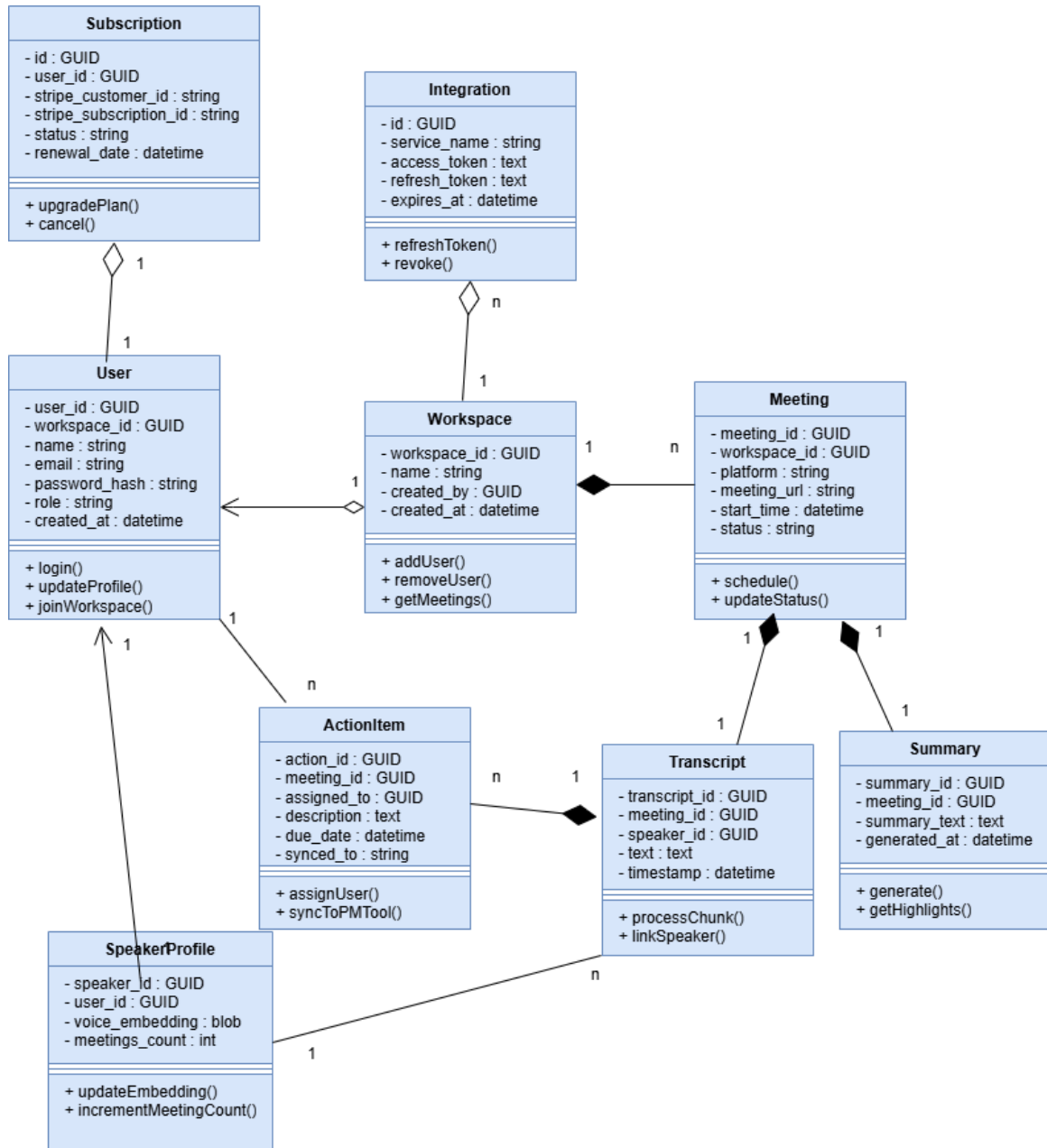
- System:Generate Transcript, Generate Summary, Extract Action Items



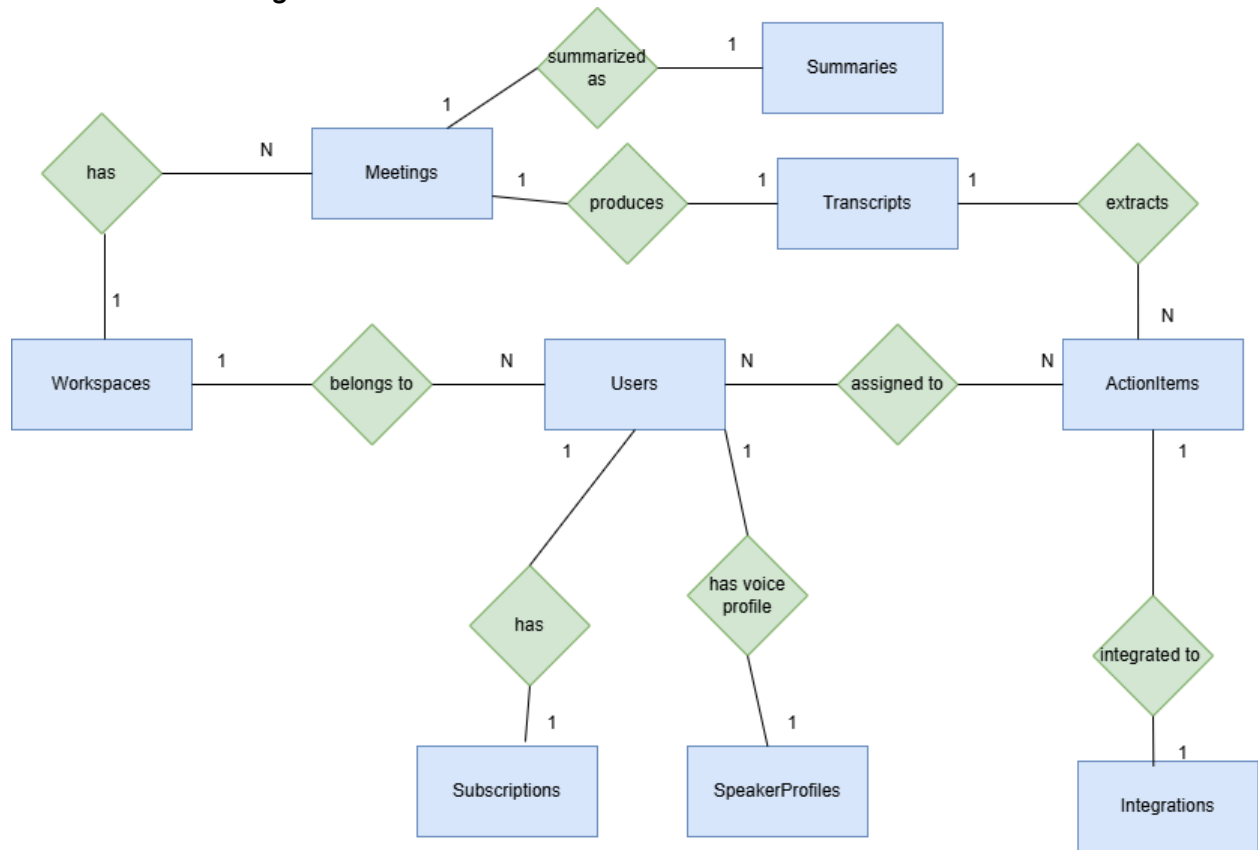
5.2. Domain Model

5.2.1. Class Diagram

- User → belongs to Workspace
- Workspace → contains Meetings
- Meeting → has Transcript
- Meeting → has Summary
- Summary → generates ActionItems
- Meeting → linked to SpeakerProfiles



5.2.2. *ER Diagram*



5.3. Data Flow Diagram

5.3.1. Level 0 Context Diagram

External Entities:

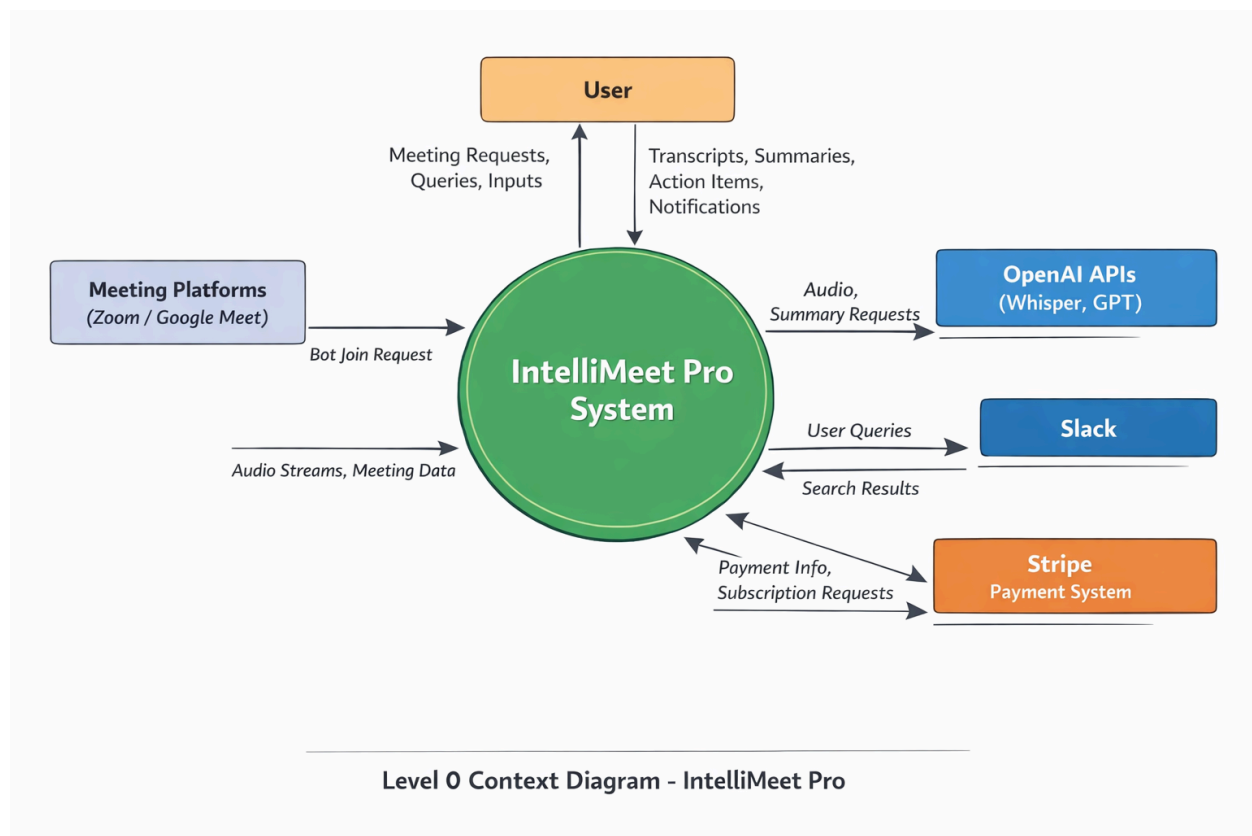
- User
- Meeting Platforms
- OpenAI APIs
- Slack
- Stripe

System:

- IntelliMeet Pro

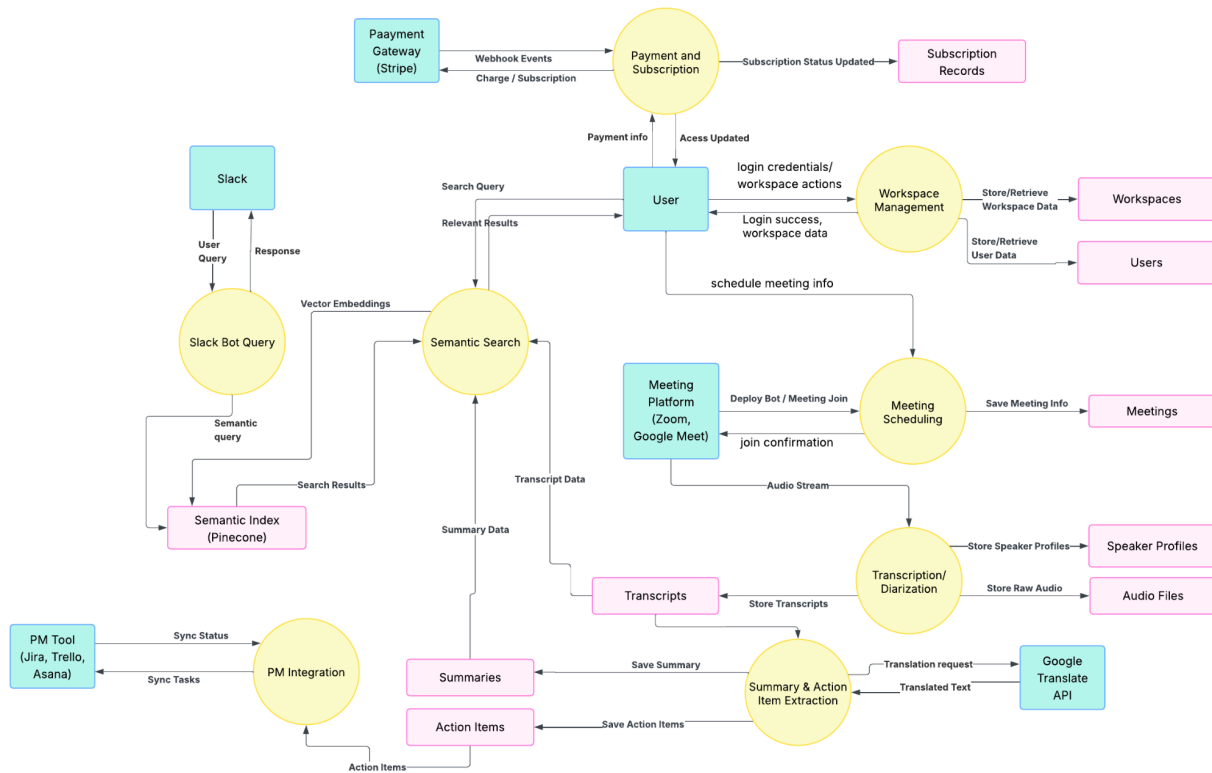
Data flows:

- Meeting data input
- Processed outputs (transcripts, summaries, tasks)



5.3.2. Level 1 DFD

This Level-1 DFD shows how IntelliMeet Pro processes meeting data end-to-end: the user authenticates, manages their workspace, and schedules meetings, which triggers the bot to join external meeting platforms. Audio from the meeting is sent to the transcription/diarization module, producing transcripts, speaker profiles, and audio files that feed into the summary and action item extraction process. Summaries and action items are stored and used by the semantic search engine, which also powers Slack bot queries. Action items are synced with external project management tools, while the payment system manages subscriptions through Stripe and updates access permissions.



6. Weakness Identification

This section focuses on identifying **internal code quality issues** within the system, rather than functional or business limitations. The analysis highlights structural, maintainability, and design-related weaknesses that impact long-term scalability and evolution.

1. Tight Coupling Between Components

The system exhibits tight coupling between core modules and external services such as AI APIs and third-party integrations. Business logic appears to directly depend on concrete implementations rather than abstractions.

2. Lack of Separation of Concerns (SoC)

Multiple responsibilities are likely handled within the same classes or services, such as API communication, business logic, and data processing.

3. Code Duplication (DRY Violation)

Similar logic patterns (e.g., API handling, validation, or data transformation) are likely repeated across different modules.

4. Inconsistent Error Handling Mechanism

Error handling is not centralized and may rely on scattered try-catch implementations.

5. Absence of Structured Input Validation Layer

Input validation is either inconsistently implemented or embedded within business logic.

6. Monolithic Code Organization

Despite having logical modules, the system is structured as a tightly integrated codebase without clear modular boundaries.

7. Hardcoded Configuration and Environment Dependencies

Configuration values such as API endpoints, keys, or system parameters may be embedded directly in the code.

8. Poor Naming Conventions and Readability Issues

Code readability may be affected by inconsistent naming conventions and unclear method or variable names.

7. *Re-Engineering Proposal*

1. Introduce Abstraction Layers for External Dependencies

To reduce tight coupling, external services should be accessed through well-defined interfaces.

Proposed Solution:

- Implement **interface-based service layers**
- Apply the **Adapter Pattern** for third-party integrations
- Use **Dependency Injection (DI)**

Benefit:

- Improves testability and flexibility
- Enables easy replacement of external services

2. Enforce Separation of Concerns

Refactor the codebase into clearly defined layers.

Proposed Structure:

- Controller Layer (request handling)
- Service Layer (business logic)
- Repository Layer (data access)

Benefit:

- Enhances modularity
- Simplifies maintenance and debugging

3. Eliminate Code Duplication

Refactor repeated logic into reusable components.

Proposed Solution:

- Extract common utilities and helper classes
- Introduce shared service modules

Benefit:

- Reduces redundancy
- Ensures consistency across the system

4. Implement Centralized Error Handling

Introduce a unified error-handling mechanism.

Proposed Solution:

- Global exception handling middleware
- Standardized error response format

Benefit:

- Improves consistency and reliability
- Simplifies debugging

5. Introduce a Dedicated Validation Layer

Separate validation logic from business logic.

Proposed Solution:

- Use **DTOs (Data Transfer Objects)**
- Integrate validation frameworks

Benefit:

- Improves security and data integrity
- Keeps business logic clean

6. Modularize the Codebase

Restructure the system into independent modules.

Proposed Solution:

- Define clear module boundaries
- Optionally prepare for microservices architecture

Benefit:

- Enhances scalability
- Enables independent development

7. Externalize Configuration Management

Remove hardcoded values from the codebase.

Proposed Solution:

- Use environment variables
- Centralized configuration files

Benefit:

- Improves security
- Supports multiple deployment environments

8. Improve Code Readability and Standards

Adopt clean coding practices across the project.

Proposed Solution:

- Enforce naming conventions
- Apply consistent formatting standards
- Conduct code reviews

Benefit:

- Enhances maintainability
- Reduces onboarding time for new developers

8. Code modifications

8.1. Hardcoded Configuration → Externalized

Before:

```
app = FastAPI()

device = "cuda"
compute_type = "int8"
batch_size = 16

HF_TOKEN = "hf_..."

print(f"Loading WhisperX ASR model on {device} ...")
asr_model = whisperx.load_model("large-v2", device, compute_type=compute_type)

print("Loading diarization model (pyannote)...")
diarize_model = DiarizationPipeline(
    use_auth_token=HF_TOKEN,
    device=device,
)
```

After:

```
HF_TOKEN = os.environ.get("HF_TOKEN", "")
if not HF_TOKEN:
    raise RuntimeError("HF_TOKEN environment variable is required but not set.")

DEVICE = os.environ.get("WHISPERX_DEVICE", "cuda" if torch.cuda.is_available() else "cpu")
COMPUTE_TYPE = os.environ.get("WHISPERX_COMPUTE_TYPE", "int8")
BATCH_SIZE = int(os.environ.get("WHISPERX_BATCH_SIZE", "16"))
ASR_MODEL = os.environ.get("WHISPERX_MODEL", "large-v2")
```

A live HuggingFace API token was hardcoded in source code which is a security vulnerability. Any developer with repo access could steal it. Externalizing it means the secret never enters version control. It also makes the service deployable across environments (CPU dev machine vs GPU production) without code changes.

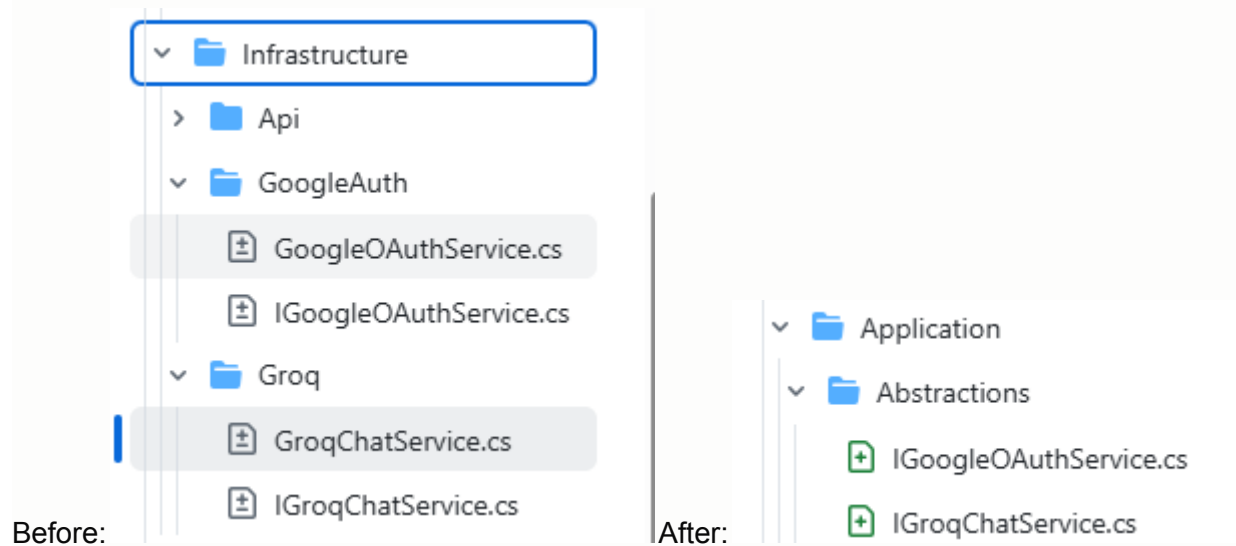
8.2. Tight Coupling / Layer Violation → Interface-Based Abstraction

New file: Application/Abstractions/IGroqChatService.cs

New file: Application/Abstractions/IGoogleOAuthService.cs

What changed: Both interfaces previously lived in the Infrastructure layer (Infrastructure.Groq and Infrastructure.GoogleAuth). They were moved to Application.Abstactions. The Infrastructure stub files were replaced with a comment pointing to the new location.

What was updated: MeetingGroqAnalysisService.cs removed using IntelliMeet.Backend.Infrastructure.Groq. GoogleAuthController.cs changed using IntelliMeet.Backend.Infrastructure.GoogleAuth to using IntelliMeet.Backend.Application.Abstractions. GoogleOAuthService.cs added using IntelliMeet.Backend.Application.Abstractions.



Benefit: The Application layer was importing from Infrastructure which is a direct violation of Clean Architecture's dependency rule (inner layers must not depend on outer layers). Fixing this means the Application layer now depends only on its own abstractions, making it independently testable and replaceable (e.g., swap Groq for OpenAI without touching any service logic).

8.3. Separation of Concerns → DTO Extraction

New file: Application/DTOs/GroqDtos.cs

What changed: GroqAnalysisDto and GroqActionItemDto were private nested classes buried inside MeetingGroqAnalysisService. They were extracted into a dedicated public DTO file.

Before:

```
...t.Backend/Application/Services/MeetingGroqAnalysisService.cs
426 -     private sealed class GroqAnalysisDto
427 -     {
428 -         [JsonPropertyName("summary")]
429 -         public string? Summary { get; set; }
430 -
431 -         [JsonPropertyName("keyPoints")]
432 -         public List<string>? KeyPoints { get; set; }
433 -
434 -         [JsonPropertyName("actionItems")]
435 -         public List<GroqActionItemDto>? ActionItems { get; set; }
436 -     }
437 -
438 -     private sealed class GroqActionItemDto
439 -     {
440 -         [JsonPropertyName("owner")]
441 -         public string? Owner { get; set; }
442 -     }
```

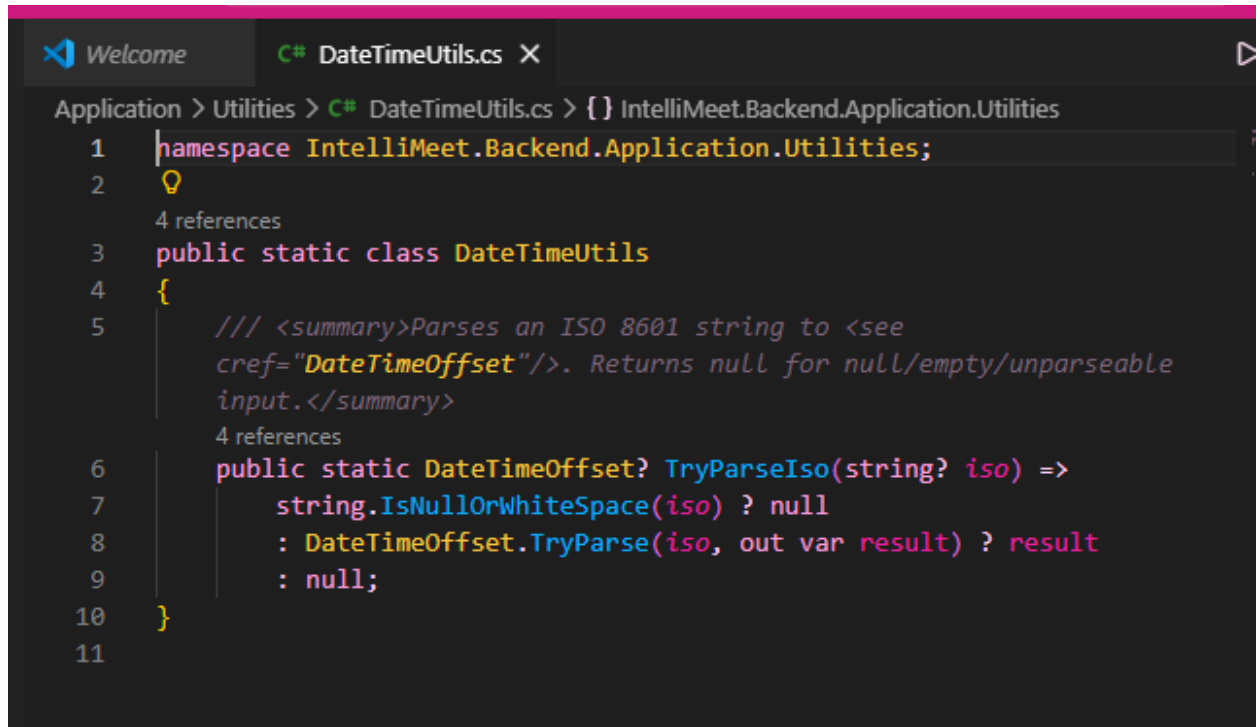
After:

```
IntelliMeet.Backend/Application/DTOs/GroqDtos.cs
... @@ -0,0 +1,27 @@
1 + using System.Text.Json.Serialization;
2 +
3 + namespace IntelliMeet.Backend.Application.DTOs;
4 +
5 + public sealed class GroqAnalysisDto
6 + {
7 +     [JsonPropertyName("summary")]
8 +     public string? Summary { get; set; }
9 +
10 +     [JsonPropertyName("keyPoints")]
11 +     public List<string>? KeyPoints { get; set; }
12 +
13 +     [JsonPropertyName("actionItems")]
14 +     public List<GroqActionItemDto>? ActionItems { get; set; }
15 + }
16 +
17 + public sealed class GroqActionItemDto
18 + {
```

Benefit: Data transfer objects are data structures, not service implementation details. Nesting them inside a service class conflated two responsibilities. Extracting them makes them reusable, discoverable, and independently testable.

8.4. DRY Violation → Shared Utility

New file:



```
1 namespace IntelliMeet.Backend.Application.Utilities;
2
3 public static class DateTimeUtils
4 {
5     /// <summary>Parses an ISO 8601 string to <see
6     cref="DateTimeOffset"/>. Returns null for null/empty/unparseable
7     input.</summary>
8     public static DateTimeOffset? TryParseIso(string? iso) =>
9         string.IsNullOrEmpty(iso) ? null
10        : DateTimeOffset.TryParse(iso, out var result) ? result
11        : null;
```

```

...Meet.Backend/Application/Services/CalendarWorkflowService.cs
310 - var start = Parse(startRow) ?? existing?.StartUtc ?? now;
311 - var end = Parse(endRow) ?? existing?.EndUtc ?? start.AddHours(1);
311 + var start = DateTimeUtils.TryParseIso(startRow) ?? existing?.StartUtc ?? now;
312 + var end = DateTimeUtils.TryParseIso(endRow) ?? existing?.EndUtc ?? start.AddHours(1);
312 313 var isCancelled = e.IsCancelled == true
313 314 || string.Equals(e.Status, "cancelled", StringComparison.OrdinalIgnoreCase)
314 315 || (string.IsNullOrEmpty(e.Status) && e.IsCancelled is null && (existing?.IsCancelled ?? false));
@@ -331,13 +332,6 @@ private void UpsertEventsFromApi(Guid connectionId, IReadOnlyList<CalendarEventA
331 332 }
332 333 }
333 334
334 - private static DateTimeOffset? Parse(string? iso)
335 - {
336 -     if (string.IsNullOrEmpty(iso))
337 -         return null;
338 -     return DateTimeOffset.TryParse(iso, out var d) ? d : null;
339 - }
340 -

```

Benefit: The exact same ISO date parsing logic existed as separate private methods in two unrelated service classes. Any bug fix or behavior change would need to be applied in two places. The shared utility eliminates that duplication.

8.5. *Inconsistent Error Handling*

IntelliMeet.Backend/Infrastructure/Api/AppExceptionHandler.cs

```
IntelliMeet.Backend/Infrastructure/Api/AppExceptionHandler.cs

@@ -11,14 +11,15 @@ public sealed class AppExceptionHandler : IExceptionHandler

11 11     public async ValueTask<bool> TryHandleAsync(HttpContext httpContext, Exception exception, Cancel
12 12     {
13 13         _logger.LogError(exception, "Unhandled exception");
14 -     (int status, string title) = exception switch
14 +     (int statusCode, string title) = exception switch
15 15     {
16 16         KeyNotFoundException => (StatusCodes.Status404NotFound, exception.Message),
17 17         ArgumentException => (StatusCodes.Status400BadRequest, exception.Message),
18 18         InvalidOperationException => (StatusCodes.Status400BadRequest, exception.Message),
19 +         UnauthorizedAccessException => (StatusCodes.Status401Unauthorized, exception.Message),
19 20         _ => (StatusCodes.Status500InternalServerError, "An unexpected error occurred.")
20 21     };
21 -     httpContext.Response.StatusCode = status;
22 +     httpContext.Response.StatusCode = statusCode;
22 23     await httpContext.Response.WriteAsJsonAsync(new { success = false, error = title },
        cancellationToken).ConfigureAwait(false);
23 24     return true;
24 25     }
```

What changed: Added `UnauthorizedAccessException => (StatusCodes.Status401Unauthorized, exception.Message)` to the exception-to-status-code switch. Renamed local variable `status` → `statusCode` for clarity.

Benefit: `UnauthorizedAccessException` was previously falling through to the catch-all 500 handler, returning an incorrect and misleading HTTP status code to API clients.

IntelliMeet.Backend/Controllers/MeetingsApiController.cs

```
IntelliMeet.Backend/Controllers/MeetingsApiController.cs +2 -13
@@ -69,18 +69,7 @@ public async Task<ActionResult<ActionItemDto>> AssignTask(Guid id, [FromBody] As
69 69 [HttpPost("{id:guid}/groq-analyze")]
70 70 + public async Task<ActionResult> GroqAnalyze(Guid id, CancellationToken ct)
71 71 {
72 - try
73 - {
74 -     await _groqAnalysis.AnalyzeAndPersistAsync(id, ct).ConfigureAwait(false);
75 -     return Ok(new { ok = true });
76 - }
77 - catch (KeyNotFoundException)
78 - {
79 -     return NotFound();
80 - }
81 - catch (InvalidOperationException ex)
82 - {
83 -     return BadRequest(new { success = false, error = ex.Message });
84 - }
72 + await _groqAnalysis.AnalyzeAndPersistAsync(id, ct).ConfigureAwait(false);
73 + return Ok(new { ok = true });
```

What changed: Removed a manual try/catch block in GroqAnalyze that caught `KeyNotFoundException` (returning 404) and `InvalidOperationException` (returning 400). The method now simply calls the service and returns Ok.

Benefit: The global `AppExceptionHandler` already handles both of those exception types with the same mappings. The controller-level catch was duplicating that logic, creating two places where the same exception could be handled differently if one was ever updated without updating the other.

8.6. Absent Validation Layer → DTO Annotations

Application/DTOs/CalendarDtos.cs

IntelliMeet.Backend/Application/DTOs/CalendarDtos.cs		
8	+	[Required]
8	9	public Guid UserId { get; set; }
10	+	
9	11	public CalendarProvider Provider { get; set; } = CalendarProvider.Google
10	12	
11	13	/// <summary>Optional when server has GoogleOAuth ClientId/Secret config
14	+	[MaxLength(500)]
12	15	public string? OAuthClientId { get; set; }
13	16	
17	+	[MaxLength(500)]
14	18	public string? OAuthClientSecret { get; set; }
15	19	
16	20	[Required]
21	+	[MaxLength(2000)]
17	22	public string OAuthRefreshToken { get; set; } = string.Empty;
18	23	
24	+	[MaxLength(200)]
19	25	public string? OAuthTenantId { get; set; }
20	26	
21	27	[Required]
28	+	[MaxLength(500)]

- ConnectCalendarRequestDto.UserId: added [Required]
- OAuthClientId, OAuthClientSecret: added [MaxLength(500)]
- OAuthRefreshToken: added [MaxLength(2000)]
- OAuthTenantId: added [MaxLength(200)]
- RawCalendarId: added [MaxLength(500)]
- ScheduleCalendarBotRequestDto — EventId, BotName: added [MaxLength(200)]; SeriesId: [MaxLength(200)]; RecordingMode: [MaxLength(100)]

Application/DTOs/MeetingDtos.cs

IntelliMeet.Backend/Application/DTOs/MeetingDtos.cs

↑

@@ -3,6 +3,7 @@

3

3

4

4

namespace IntelliMeet.Backend.Application.DTOs;

5

5

6

+

6

7

public sealed class MeetingListItemDto

7

8

{

8

9

public Guid Id { get; init; }

↓

@@ -89,16 +90,23 @@ public sealed class ActionItemDto

↑

89

90

public sealed class AssignTaskRequestDto

90

91

{

91

92

[Required]

93

+

[MaxLength(500)]

92

94

public string Title { get; set; } = string.Empty;

93

95

96

+

[MaxLength(2000)]

94

97

public string? Description { get; set; }

98

+

99

+

[MaxLength(200)]

95

100

public string? Assignee { get; set; }

101

+

- AssignTaskRequestDto.Title: added [MaxLength(500)]; Description: [MaxLength(2000)]; Assignee: [MaxLength(200)]
- ConvertActionItemToTodoRequestDto.TODOType: added [MaxLength(100)]

Application/DTOs/TodoDtos.cs

```

IntelliMeet.Backend/Application/DTOs/ToDoDtos.cs
@@ -18,18 +18,26 @@ public sealed class ToDoItemDto
18 18 public sealed class CreateToDoRequestDto
19 19 {
20 20     [Required]
21 + [MaxLength(500)]
21 22     public string Title { get; set; } = string.Empty;
22 23
24 + [MaxLength(2000)]
23 25     public string? Description { get; set; }
26 +
27 + [MaxLength(100)]
24 28     public string Type { get; set; } = "manual";
29 +

```

- CreateToDoRequestDto.Title: added [MaxLength(500)]; Description: [MaxLength(2000)]; Type: [MaxLength(100)]
- PatchToDoRequestDto.Title: added [MaxLength(500)]

Benefit: Without length constraints, an attacker or buggy client can send arbitrarily long strings, causing database column overflow errors or excessive memory use. ASP.NET Core's model validation layer rejects invalid requests at the controller boundary with a 400 Bad Request before they reach service or database code.

8.7. Poor Naming Conventions

GroqChatService.cs

Before	After
req	request
resp	response
parsed	completion
body	chatRequest

MeetingGroqAnalysisService.cs

Before	After
plain	transcriptText
mbJson	meetingBaasContextJson
system	systemPrompt
user	userContent
loop var a	actionItem
loop var it	item

Benefit: Abbreviated names like `req`, `resp`, `plain`, and single-letter loop variables force the reader to mentally decode what the variable represents. Descriptive names make intent self-evident, reducing cognitive load and the chance of misuse (e.g., confusing `system` with the system prompt vs. another string).

9. Conclusion

This project successfully applied software re-engineering principles to IntelliMeet Pro by analyzing its architecture, recovering its design, identifying weaknesses, and proposing improvements.

The system demonstrates strong capabilities in AI-driven meeting automation but suffers from complexity, dependency risks, and scalability challenges. The proposed re-engineering solutions aim to enhance performance, reliability, and user experience.

Through this exercise, the importance of reverse engineering and system improvement in real-world software systems is clearly demonstrated.